TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING (IOE), PURWANCHAL CAMPUS

DEPARTMENT OF COMPUTER AND ELECTRONICS ENGINEERING

DSAP LAB ASSIGNMENT - 2

SIGNAL OPERATIONS ON BASIC SIGNALS

BY

Tilak Thapa [PUR079BCT094]

29 JUNE

OBJECTIVE

The objective of this lab assignment is to implement common signal operations on a basic signal generated in Lab Exercise 1 and display the results graphically. Each operation is implemented using a separate Python function.

SOFTWARE AND LIBRARIES USED

- Programming language: Python
- Libraries used: NumPy and Matplotlib
- Purpose of NumPy: numerical signal computation
- Purpose of Matplotlib: graphical representation of signal operations

BASE SIGNAL USED

A rectangular signal from Lab Exercise 1 is used as the base signal. It has value 1 for a selected width and value 0 outside that interval. This signal is suitable for demonstrating amplitude scaling, time scaling, shifting, flipping, multiplication, convolution, and correlation. The original signal is shown together with the operation results in the combined output figure.

THEORY AND IMPLEMENTATION

The following subsections describe each signal operation. The code snippets show separate Python functions used for the operations. NumPy is imported as `np` in the program.

1. AMPLITUDE SCALING

Amplitude scaling changes the height or strength of a signal without changing its time position.

$$y[n] = Ax[n]$$

Here, $A = 2$.

```
# Input: signal x, scaling factor A
# Output: amplitude-scaled signal
def amplitude_scaling(x, A):
    y = A * x
    return y
```

The function returns the amplitude-scaled signal values.

2. TIME SCALING

Time scaling compresses or expands a signal along the time axis.

$$y[n] = x[an]$$

Here, $a = 2$.

```
# Input: n, signal x, scaling factor a
# Output: time-scaled signal values
def time_scaling(n, x, a):
    lookup = dict(zip(n, x, strict=True))
    y = np.array([lookup.get(a * index, 0) for index in n])
    return y
```

The function returns the time-scaled signal values.

3. TIME SHIFTING: TIME DELAY

Time delay shifts a signal to the right by a selected number of samples.

$$y[n] = x[n - k]$$

Here, $k = 5$.

```
# Input: n, signal x, delay value k
# Output: delayed signal values
def time_delay(n, x, k):
    lookup = dict(zip(n, x, strict=True))
    y = np.array([lookup.get(index - k, 0) for index in n])
    return y
```

The function returns the delayed signal values.

4. TIME SHIFTING: TIME ADVANCE

Time advance shifts a signal to the left by a selected number of samples.

$$y[n] = x[n + k]$$

Here, $k = 5$.

```
# Input: n, signal x, advance value k
# Output: advanced signal values
def time_advance(n, x, k):
    lookup = dict(zip(n, x, strict=True))
```

```
    y = np.array([lookup.get(index + k, 0) for index in n])
    return y
```

The function returns the advanced signal values.

5. TIME FLIPPING

Time flipping reverses a signal around the vertical axis.

$$y[n] = x[-n]$$

```
# Input: n, signal x
# Output: time-flipped signal values
def time_flipping(n, x):
    lookup = dict(zip(n, x, strict=True))
    y = np.array([lookup.get(-index, 0) for index in n])
    return y
```

The function returns the time-flipped signal values.

6. MULTIPLICATION IN THE TIME DOMAIN

Multiplication in the time domain multiplies two signals sample by sample.

$$y[n] = x_1[n] \times x_2[n]$$

The base rectangular signal and a delayed rectangular signal are used.

```
# Input: two signals x1 and x2
# Output: sample-wise multiplied signal
def time_domain_multiplication(x1, x2):
    y = x1 * x2
    return y
```

The function returns the sample-wise product of two signals.

7. MULTIPLICATION IN THE FREQUENCY DOMAIN

Frequency domain multiplication is performed by multiplying the FFT values of two signals.

$$Y[k] = X_1[k] \times X_2[k]$$

The inverse FFT is used to show the result in the time domain.

```
# Input: two signals x1 and x2
# Output: inverse FFT of multiplied spectra
def frequency_domain_multiplication(x1, x2):
    X1 = np.fft.fft(x1)
    X2 = np.fft.fft(x2)
    y = np.fft.ifft(X1 * X2).real
    return y
```

The function returns the time-domain signal obtained from multiplied spectra.

8. CONVOLUTION

Convolution combines two signals and is commonly used to represent the response of a system.

$$y[n] = x[n] * h[n]$$

The base rectangular signal and a delayed rectangular signal are used.

```
# Input: two signals x and h
# Output: linear convolution values
def convolution_operation(x, h):
    y = np.convolve(x, h, mode="full")
    return y
```

The function returns the linear convolution of two signals.

9. CORRELATION

Correlation measures the similarity between two signals as one signal is shifted over another.

$$r_{xy}[n] = \text{corr}(x[n], y[n])$$

The base rectangular signal and a delayed rectangular signal are used.

```
# Input: two signals x and y
# Output: correlation values
def correlation_operation(x, y):
    rxy = np.correlate(x, y, mode="full")
    return rxy
```

The function returns the correlation values of two signals.

OUTPUT

The combined output image below shows the original signal and all implemented signal operation results in one view.

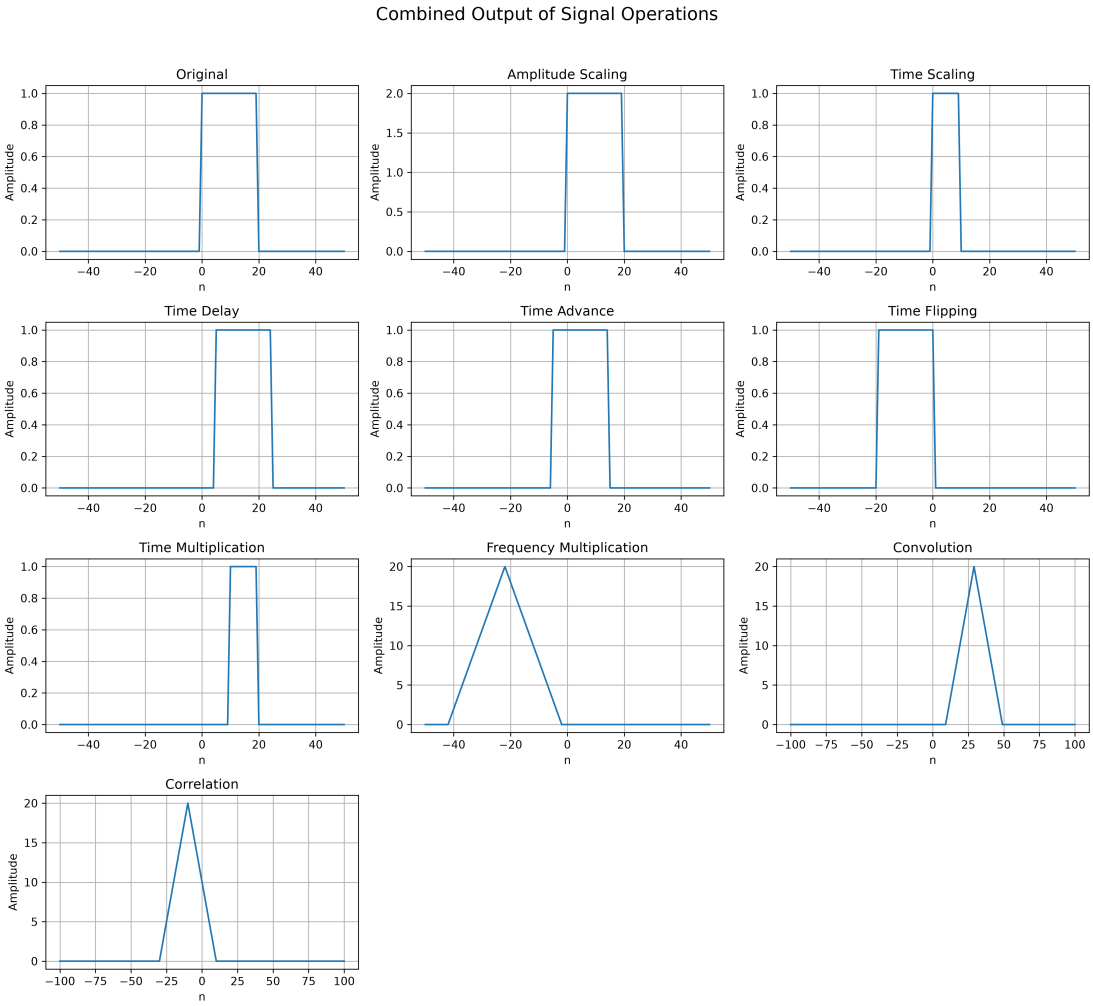

Figure 1: Combined output of all implemented signal operations

RESULT AND DISCUSSION

The output successfully demonstrates the required signal operations. Amplitude scaling changes the height of the signal. Time scaling compresses the signal along the time axis. Time delay shifts the signal to the right, while time advance shifts it to the left. Time flipping reverses the signal around the vertical axis. Multiplication in the time domain combines two signals sample by sample, and multiplication in the frequency domain combines their spectra. Convolution gives the combined response of two signals, and correlation shows the similarity between two signals.

CONCLUSION

The lab assignment was completed by implementing each signal operation in a separate Python function and displaying the output graphically. The results show the effects of amplitude scaling, time scaling, shifting, flipping, multiplication, convolution, and correlation on a basic discrete-time signal.